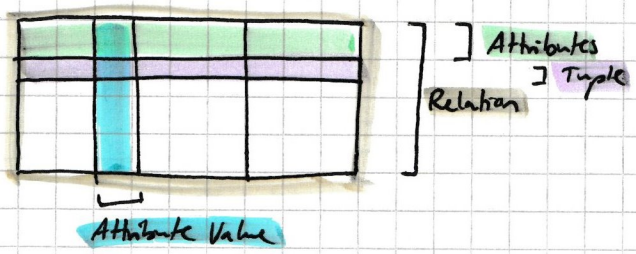
Chapter 1: Introduction

DB
24/01/26



Flat file databases : one singular table containing all information

↳ Disadvantages **Inconsistency** : same object is represented twice

Data redundancy : same data saved multiple times

Anomalies: **Insertion** → insertion of incomplete data

Update → modification/insertion of redundant data that needs to be kept consistent

Deletion → unintentional loss of data through ~~typo~~ deletion

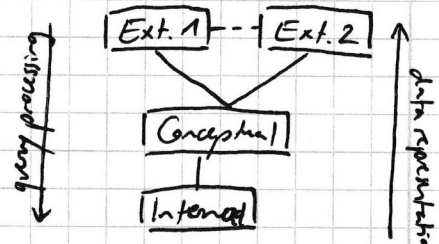
Schema Architecture

> purpose-related form of data representation (schema), i.e. relation attributes

Conceptual schema: representation in tables (relations) data definition

Internal schema: internal organisation of the data file organisation and access paths

External schema: different schema per use case view definition



[redacted] **DDL:** Data Definition Language → changes to schema (structure)

DML: Data Manipulation Language → changes to content (data set)

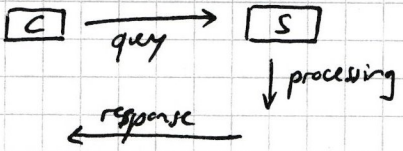
Data Independence

Why?

→ stability: **physical**; changes to file orga and access path have no effect on conceptual schema

logical; changes to conceptual schema and certain ext. schema have no effect on other ext. schema and appl. programmes.

Client-Server Model:



Separation of application functionalities: **Presentation**, **Logic**, **Database Interface**

Data Model defines **static properties** (structure of data), **dynamic properties** (operations)

integrity conditions (constraints)

Chapter 2: Relational Algebra

Integrity Constraints

Key Constraint > no two tuples depend on the same key value

Terms/Concepts: Keys

- **Key Attribute**: uniquely identifies tuple, can be part of combination
- **Key Candidates**: all attributes (incl. combinations) that could be primary key for a relation
- **Primary Key**: chosen key attribute that determines whole relation
  - → **Primary Attribute**: attribute that is part of prime key
- **Superkey**: all attribute (combinations) that are key candidate and could be key
- **Surrogate Key**: automatically generated numeric key, created with sole purpose of being key, also called artificial key

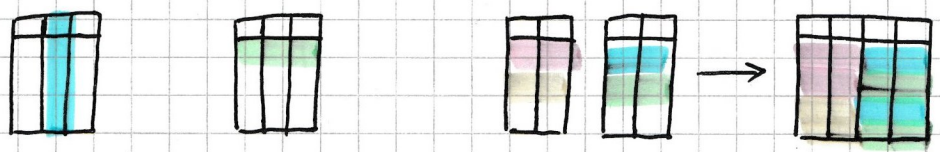
opposite: natural key, consists of attributes already present in relation

Foreign Key: key from another table that is referenced, [redacted] usually part of key

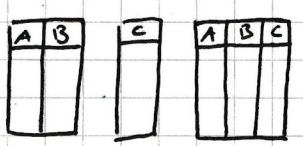
Referential Integrity > one set of attri. values is subset of other set of attri. values, i.e. foreign keys

Relational Algebra: Overview

Projection π **Selection** σ **Cross Product** $\times$



Union



Difference: $R - S$
↳ "R without vs tuples that are also present in S"

Intersection: $R \cap S \triangleq R - (R - S)$
↳ "tuples that are both in R and S"

Chapter 3: SQL Introduction

```
create table table_name [redacted] (
    attr_name data_type [redacted] (NOT NULL),
    foreign key (key_name) references other_table (key),
    primary key (attri_name1, ..., attri_nameN),
);
```

data types:
int, float
varchar (size)
int identity (1,1)
⋮

[redacted]

```
drop table table_name;
```

DB
24/01/26
contd.

Chapt. 3 contd.

```
alter table table_name add field_name datatype;

alter table table_name drop column field_name;

alter table table_name add constraint no_negative_values check (attr_name >0);

alter table table_name drop constraint no_negative_values;
```

```
insert into table_name (attr_name1, attr_name2) values
    (value1, value2),
    (value1, value2);
```

```
update table_name set attr_name = new_value where [redacted] attr_condition
```

Propagating foreign key changes

foreignkey_name data_type [redacted] foreign key references table(attr)

on delete set null on update cascade

↑

i.e. propagate change to other tables

[redacted]

Verification modes of conditions: on update / on delete

set

> cascade: changes of referenced value are transfered to local
if referenced tuple is removed, local tuple is also removed

> set null: if referenced tuple removed or changed, attr value of local tuple is set to null

> set default: if —"—, attr value is set to default value

> no action: default value, nothing is done locally

```
select
        attr_name,
        cast(attr as varchar) + '%',
        case                          condition
                when [redacted] then [redacted] result
                else [redacted] default_result
        end as 'Attribute Name',
        substring(attr, num_chars, start_pos)          e.g substring(name, 1, 1)
        charindex('string', attr) as 'Position of "string"',
        'It is now' + cast(convert(time, current_timestamp) as varchar(5)) + 'O'clock' as Uhrzeit
from    table as t
        inner join table2 as t2 on condition          inner join ≙ join
where   attr not like 'string' and
        attr not in (selection) and
        attr_numerical >= all (select attr_numerical from table)
```

```
delete from table where condition;
delete from table;
truncate table table_name;        } empties table (schema remains)
```

```
SFW block
union (all)
SFW block
```

→ all: includes duplicates, default is [redacted] distinct

must [redacted] have same schema as above SFW block (i.e. same attribute selection)

also: intersect, except

```
SF block
where attr in/not in (SFW block)
where name like 'string%'
```

→ selection range via subquery

→ % is wildcard and can be filled however A% would catch A1, AB, A2, etc.

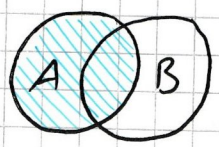
```
case
    when [redacted] condition then result
    :
    else default_result
[redacted]
end as 'Attribute Name'
```

- [redacted] should/can replace attr in select block or also be used in group by, returns varchar

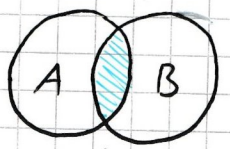
```
S block
from table join table2 on condition
from table [redacted] left join table2 on condition
```

→ returns tuples that are in both

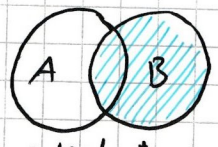
→ add tuples from table2 to table1




```
select attributes
from table A
left join table B
on A.key = B.key
```

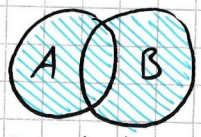



```
select *
from A
inner join B
on A.key = B.key
```

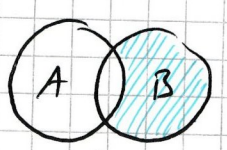



```
select *
from A
right join B
on A.key = B.key
```

```
select -11-
from A
left join B
on A.key = B.key
where B.key is null
```




```
Select *
from A
full outer join B
on A.key = B.key
```




```
select *
from A
right join B
on A.key = B.key
where A.key is null
```

DB
2401126 cont'd.

Chapter 4: Relational Database Design

> relation normalization is done to find the sweet spot between avoiding redundancy and avoiding exponential relation growth with low performance

> achieve data consistency and avoid anomalies

Normalization

1. Identify functional dependencies
2. Calculate the key
3. Check if attributes fulfill requirements of a normal form, preferably BCNF
4. If no: Normalize

Functional Dependency: attr is determined by another and thus functionally dependent on it (key-value relationship)

Properties of FDs

Triviality: every attribute determines itself

Reflexivity: every set of attributes determines its subset

Augmentation: every left side can be enlarged (augmented)

Decomposition: every right side can be split

Union: right sides with the same left side can be combined

Transitivity: FDs can be extended by following links

Pseudo-transitivity: $\{X \to Y,\ W, Y \to Z\} \subseteq FD_R$

$\Rightarrow W, X \to Z \in FD_R$

Splitting Algorithm

1. Split all FDs via Decomposition
2. Remove all trivial and double FDs
3. Summarize where possible using Union

Closure of FDs

> Functional Dependency Closure FD^+ refers to set of all FDs that can be derived from existing set FD

eg.: Initial FDs: $A \to B$, $B \to C$

$FD^+ = \{A \to B,\ B \to C,\ A \to C,\ A \to A,\ B \to B,\ C \to C,\ AD \to BD\}$

(transitive: $A \to C$; trivial: $A \to A$, $B \to B$, $C \to C$; augmented: $AD \to BD$)

$R = \{A, B, C, D\}$

Closure of Attributes

[redacted] FD

> set of all attributes that are functionally dependant on the given attribute (Closure Algorithm)

e.g.: $FD = \{ A \to B,\ B \to C,\ A,C \to D \}$

$\{A\}^+ = \{A, B, C, D\} \Rightarrow$ A would be key if $\{A\}^+ = R$ i.e. determines all attributes in R

- A — trivial
- $A \to B$
- $A \to B$, $B \to C$ — transitive
- $A,C \to D$ — pseudo-transitive

Member Algorithm / Membership Problem

> to determine if B is member of A, check if $B \in \{A\}^+_R$

1. $\{A\}^+_{FD}$
2. $B \in \{A\}^+_{FD} \Rightarrow A \to B \in FD \Rightarrow Member(FD, A \to B) = true$
   (case or false if $B \notin \{A\}^+_{FD}$)

Key Heuristic

1. Splitting (FD) to simplify
2. all attributes not on right side are part of key
3. check via Closure if (2) is already key
4. if not, check combination of (2) and remaining attributes

→ possible to get multiple keys (key candidates)

Cover Algorithm

Ziel: make FDs more compact

1. Splitting to simplify
2. Minimize left side by remaing augmentations (if right side can be reached without [redacted] attr, then attr is removed)
3. Remove unnecessary FDs (transitivity)
4. Summarize left side

DB

24/01/26 contd.

Normal Forms and their Constraints

1NF: every attribute has an atomic range of values

R is free of repeating groups

2NF: every attribute that is not part of a key depends on the whole key and not just on a real subset

3NF: No non-key attribute is transitively dependant on a key

left side superkey or right side prime

BCNF: every attribute set that other attributes functionally depend on is a superkey

"The key (1NF), the whole key (2NF) and nothing but the key (3NF), so help me Codd!"

Decomposition

1. Identify Key and apply Cover to FDs
2. Identify first FD that violates BCNF
3. Decompose R into $R_1 = \{A_1, \dots, A_n\}^+_{FD}$ ← Violating FD: → Key and Key Closure

$R_2 = R \setminus R_1 \cup \{A_1, \dots, A_n\}$ ← remaining attr + key from R_1

4. check dependency preservation $FD_{R_1} \cup FD_{R_2} = FD_R$
5. check BCNF, if not reached, then continue with decomposed schemas

Synthesis

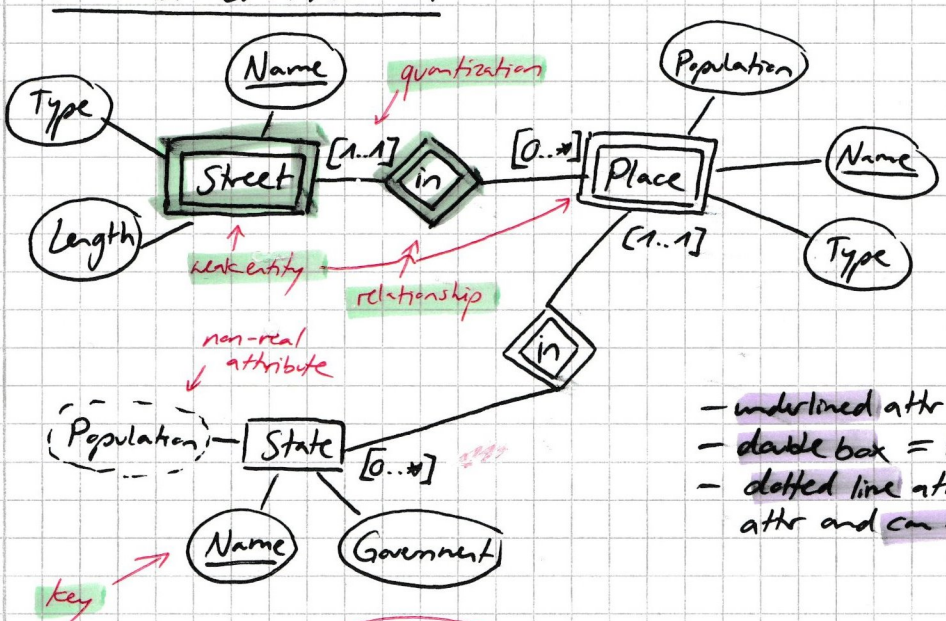
1. Calculate Cover
2. every FD → own relation, summarize right sides
3. if no superkey, then create superkey relation
4. remove unnecessary relations that are subsets of others $R'' \subset R'$

Lossy join: information is lost during join due to missing foreign key
(opposite: lossless join)

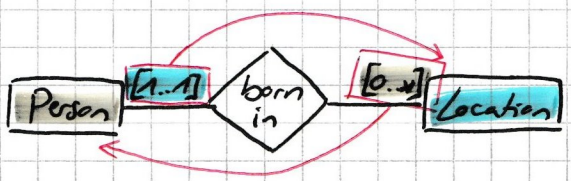
Chapter 6: Conceptional Database Design

Entity-Relationship Diagrams (ER)

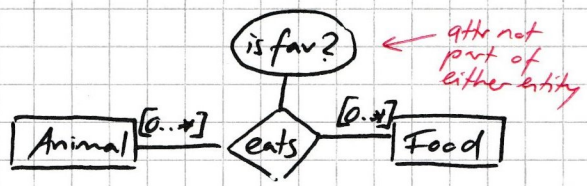
Minmax – Chen Notation

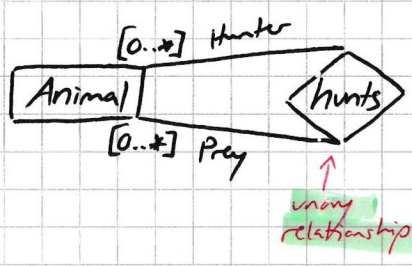


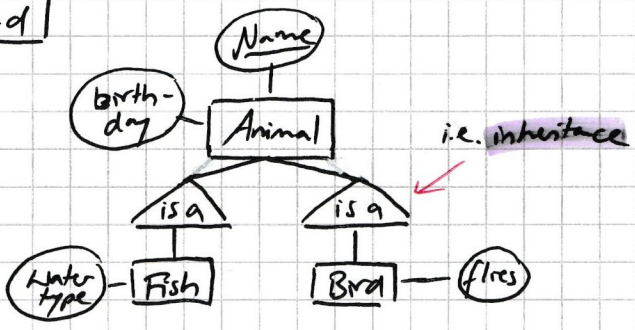
- underlined attr are key
- double box = weak entity
- dotted line attr is not real attr and can be calculated



"Every person is born in exactly one location, in every location zero or any number of people are born"







> Weak entities are [redacted] entities with keys that cannot uniquely identify them, needing a supporting relationship

ER Mapping to relational model

relationship attribute references related entity

> every entity type becomes a relation

> every relationship becomes a relation

> relationships are always merged to the 1-side (incl. attributes)

> one-to-one are merged into singular relation

> weak entities collect all other keys as foreign keys that are part of primary key

DB
24/01/26 contd.

Mapping is-a relationships

Null-Value Style: all attr in one relation, "child" attr can be null

ER style: every entity → relation, child references parent key
↑ has all child attributes

OO style: every entity → outer relation * (repeats attr) + combined children relation

[redacted]

DB
25/01/26

Chapter 7: Advanced SQL 1 – Aggregates, Outer Joins, Top-k, Sorting

SQL Core

select	projection list → attributes arithmetic operations & aggregate functions
from	relations to be used, possible renaming
where	selection conditions, join conditions [redacted] nested queries (SQL block)
group by	grouping for aggregate functions, tuple summarisation
having	selection condition for groups
order by	output order

Calculation Sequence: from, where, group by, having, select, order by

Aggregate Functions: calculate new values for an entire column

count: num of values in column or num of tuples in relation (count(*))

sum: sum of [redacted] values (numerical only) in a column

avg: arithmetic mean of values in column (numerical only) $\frac{\text{sum values}}{\text{num values}}$

max/min: largest or smallest value in a column

stdev/var: standard deviation or variance (average dist from mean) (above squared)

distinct: eliminates duplicates

all: includes duplicates (default)

> cannot nest aggregate functions (e.g. sum(avg(x)))

Ex.: select COUNT(distinct state) from customers;

select max(datediff(day, order_date, required_date))
from orders;

Joins

> Equi/Natural Join: used for key/foreign key relationships

> Theta Join: different join condition [redacted] than equality

> Semi Join: only attr of one operand appear in results

> Inner Join: intersection > Left Join: add tuples of second operand to first

> Full Outer Join: combines all rows

Sorting

> order by <attr_list> asc/desc

↳ is last operation of query, sorting attr must appear in select

> top-k returns fraction of query results according to ranking func

→ select top(3) * from orders order by order_date desc;

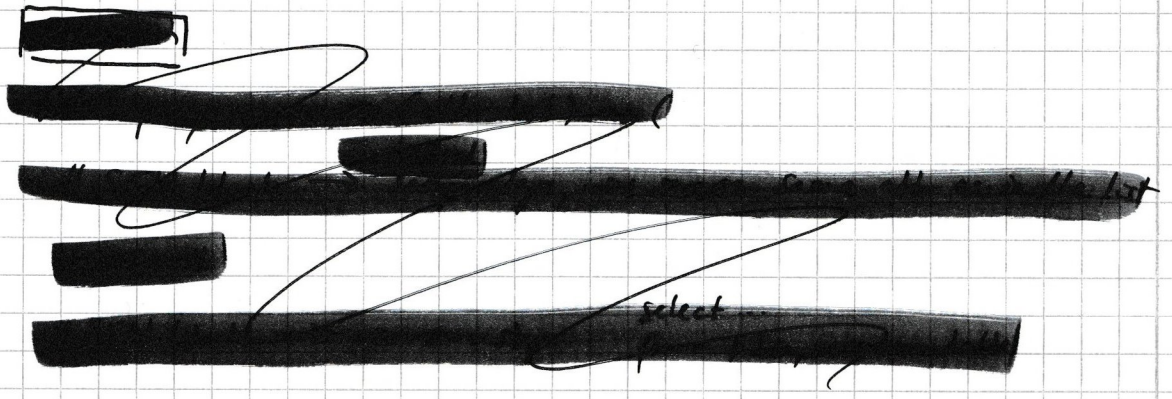
Chapter 7: Recursion and Division

Named Queries

> can be referenced in subsequent queries

```
with query_name (column_attr_list) as (
    // query_expression
);
select * from query_name;
```

[redacted]



DB

25/01/26 contd.

Recursion

```
with recursion_table [redacted] (attr_list) as (
    select ... (same attr as in attr_list)
    from table
    where ...                                   Initialisation Setup step
    union all                                   Recursive Part
    select ... (same attr)
    from table, recursion_table                 Recursion step
    where recursion_condition
)
query_expression                                Non-recursive Part
```

> essentially creates a table with attr_list as columns and each recursion step adds a new [redacted] tuple that can then be referenced during the Recursion Step of the named query

> Recursive calculations are possible by defining a counter variable for example

e.g.

```
with rec (attr1, attr2, counter) as (
select attr1, attr2, 1          <- initialisation
F...
W...
  union all
select rec.attr1, attr2, counter+1   <- recursive calculation step
F...
W...
)
```

> Computability (= finiteness of the calculation) is requirement and hindered by possible cycles with the recursion

→ can be handled by proper query design, recursion level limit via OPTION(MAXRECURSION <X>) or cycle detection

↳ limit recursion depth via counter variable (e.g. where counter < 5)

↳ manual cycle detection via varchar "counter" (e.g. where path not like '%' + bus.dest + '%')

Division

> filtering by multiple attributes

```
select <attributes> from R1
except
select <attributes> from (
    select <attributes>, R2.* from R1, R2
    except
    select * from R1
) as x
```

all possible property combos minus all real property combos

```
create table [redacted]#properties (attributes)
insert into #properties values (prop1), (prop2), (prop3), ...
```

→ temporary table to hold properties to filter for

→ [redacted] R1 is table being filtered, R2 is filter

e.g.

```
select movie from movies
except
select movie from(
    select movie, fav_properties.property
    from movies, fav_properties
    except
    select movie, property from movies
) as properties_that_do_not_exist
```

Chapter 7: [redacted] PSM Persistent Stored Modules

- problems of client-server systems:
  - > constant change of exec control between application (client) and [redacted] DBS (server)
  - > no optimisation across multip. statements possible

> Solution: stored procedures

↳ software modules administered & executed in DBMS (procedures, functions)

↳ called from [redacted] applications and queries / action parts of triggers

Variables

```
DECLARE @local_variable [AS] data_type [= value] [;]
SET @local_variable = expression [;]
```

PB

25/01/26 contd.

Control Flow - If

```
BEGIN
    // sql_statement | statement_block
END
```

```
IF boolean_expression
  // sql_statement
[ELSE
  // sql_statement ];
```

```
CASE
   WHEN boolean_expression THEN result_expression
   ELSE default_result
END AS name
```

```
WHILE boolean_expression
  // sql_statement | BREAK | CONTINUE
```

→ BREAK : terminates innermost loop

→ CONTINUE: restarts WHILE loop

Function

→ user-defined, accepts parameters and performs a pre-defined action

→ database state [redacted] must not be changed within, function must be deterministic

```
CREATE FUNCTION function_name (@param [as] data_type [= default])

RETURNS data_type
 [AS]
BEGIN

   // function body

   RETURN scalar_expression

END [;]
```

skipped: Procedures (essentially function that can modify the database, out params)

[redacted] Cursor

```
declare cursor_name [redacted] cursor for

select attr1, ..., attrN from table order by attr asc/desc;
```

```
open cursor_name;
```
→ cursor usage, triggers generation of data

```
deallocate cursor_name;
```
→ throw away data, no garbage collection during execution

> iterating through cursor results

```
FETCH NEXT | [redacted] PRIOR | FIRST | LAST FROM cursor_name
INTO @variable_name1, @variable_name2, ...
```

> can be combined with while loop:

```
WHILE @@FETCH_STATUS =0        <- is 0 while tuples in cursor
begin
  // fetch block, other sql statements, etc
end;
```

Trigger

→ PSM script that are executed when data in tables changes

→ registered for certain event and [redacted] certain table

```
create trigger trigger_name on table
instead of | after   insert | update | delete
begin
    // if statement
    // operation that is done instead (insert/update/delete) or after
end;
```

> [redacted] instead of replaces expression and triggers defined one tuple not yet affected

> after can complement ~~DRECS~~ successful execs, tuple already affected

→ inserted table: (avail. during trigger)

	insert	delete	update
instead of	tuples from insert operation, not yet part of the database	no tuples	tuples that are about to be inserted (update = delete + insert), not yet part of DB
after	copies of inserted tuples, already part of DB	no tuples	copies of inserted tuples, already part of DB

→ deleted table: (avail. during trigger)

has copies of tuples that were changed (update) or deleted (delete)

tuples are either still part of DB (instead of) or already deleted (after)

DB

25/01/26 contd.

rollback

> undoes operation, reverts to beginning of transaction

e.g.

```
create trigger no_negative_prices on products
after insert as
begin
    if exists (select * from inserted where list_price < 0) rollback;
end
```

→ causes exception reported to caller

Chapter 7: Views and Access Control

> Views are virtual relations generated from base tables, show certain aspects or part of data

> reading is unproblematic, insert/update/delete operations are due to possible data inconsistencies, integrity errors

Projection Views: address only one relation [redacted]

→ changes possible if invisible attributes can be null and operation doesnot cause inconsistencies

Effect Conformance: user can see effect of operation as though it had directly been executed on the viewing relation

→ [redacted] violated via tuple migration: added tuple falls out of view due to incompatible [redacted] condition where

→ with check option regulates operation execs [redacted] by checking where condition

Join Views: contains select with joins, operations cannot be translated directly

Minimality: base database should be modified minimally to achieve desired effect

Consistency preservation: updating a view must not lead to integrity violations in the base database

Data protection: if view introduced for data protection, consciously hidden parts must not be affected by updates to the view

Aggregation Views: contains select with aggregates and joins, no translation possible

Access Control

Rights Assignment

```
grant <Rights>      (i.e. select, insert, update, delete)
on    <Table>
to    <Userlist>    (authorisation identifier, also public, groups)
  [with grant option]  (allows user to pass on rights)
```

```
revoke <Rights>
on     <Table>
from   <Userlist>
       [restrict | cascade]
```

restrict: terminates revoke if right were already passed on

cascade: revoked rights from all users (incl. granted ones)

```
create role role_name;
```
→ roles that rights can be granted to

Example for public interface:

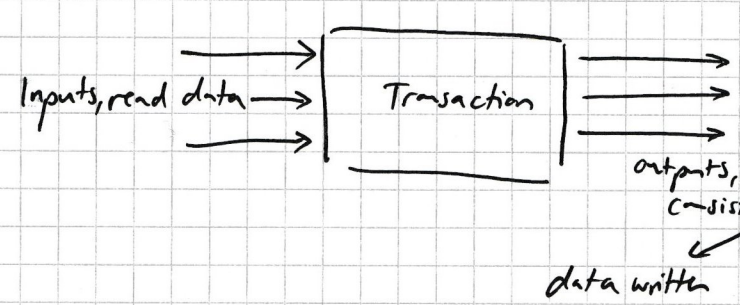
```
create login public_web_user with password = '';
create user public_web_user for login public_web_user;
```

Chapter 8: Transaction Management

Definition 'Transaction': a sequence of operations (actions) that transfer the database from a consistent state to a (possibly modified) consistent state, following the ACID principle

Semantic Integrity: correct (consistent) DB state after end of transaction

Processing Integrity: avoid errors caused by "simultaneous" access of multiple users to the same data



DB

25/01/26 contd.

Two laws of Concurrency

1. Concurrent or simultaneous exec of tasks should not cause programmes to run incorrectly (Isolation)
2. Concurrent exec of tasks should not be significantly slower than sequential exec.

ACID properties

Atomicity: transaction is either executed fully or not at all

Consistency / Integrity Preservation: DB is in consistent state before and after completed transaction

Isolation: users working with DB should have impression they are only ones working with DB

Durability: after successful transaction, result of transaction must be stored "permanently" in DB

Serial Execution U1 before U2: modification correct but transaction exec time significantly longer

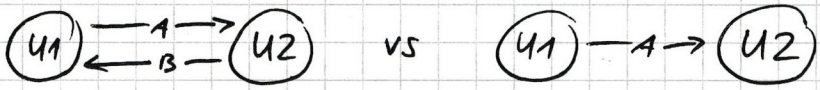
Mixed Execution of U1 and U2: modification possibly incorrect, acceptable wait time however

Data Allocation

Static Allocation: transaction specifies at design time which objects it has read/write access to

Dynamic Allocation: objects are allocated at runtime

> Mixed execution is conflict free if dependency graph has no cycles



write → write dependency: lost update → value not correctly updated

write → read depend.: dirty read → value that has not been commited yet was read

```
read(A,x)
write(x,A)
```

unrepeatable read → value in row is retrieved twice but has different values

phantom read → two identical queries are exec with different results

Phantom Problem: does the data set exist or not?

Serialisability

definition: interleaved exec of multiple transacts are serialisable if its effect is identical to the effect of a (randomly selected) serial exec of these transactions

read/write model: Transaction T is a finite sequence of operations (steps) p_i of the form $r(x_i)$ or $w(x_i)$

complete transaction has either abort a or commit c as last step $T = p_1 \dots p_1 a$ $T = p_1 \dots p_1 c$

schedule: [redacted] complete schedule is sequence of DB operations where all ops belong to complete transact and all ops of those transacts [redacted] occur in the same relative order in the schedule as in the transact

e.g.: $r_1(x) r_2(x) w_1(x) r_2(y) a_1 w_2(y) c_2$

$r_1(x) r_2(x) w_1(x)$ — schedule

$r_1(x) r_2(x) w_1(x) r_2(y) a_1 w_2(y) c_2$ — complete schedule

↳ **serial schedule:** $s := T_{p(1)} \dots T_{p(n)}$ i.e. two transact schedules in sequence [redacted] order

correctness criterion: schedule s is correct if effect is same as random serial schedule s' ($s \approx s'$)

Locking Protocols

read lock of obj x: [redacted] $rl(x)$

write lock of obj x: $wl(x)$

locking rules: write access only possible with lock

read access only possible with lock

before a commit, all locks must be released

Deadlocks: occurs when two transactions block one another via locks